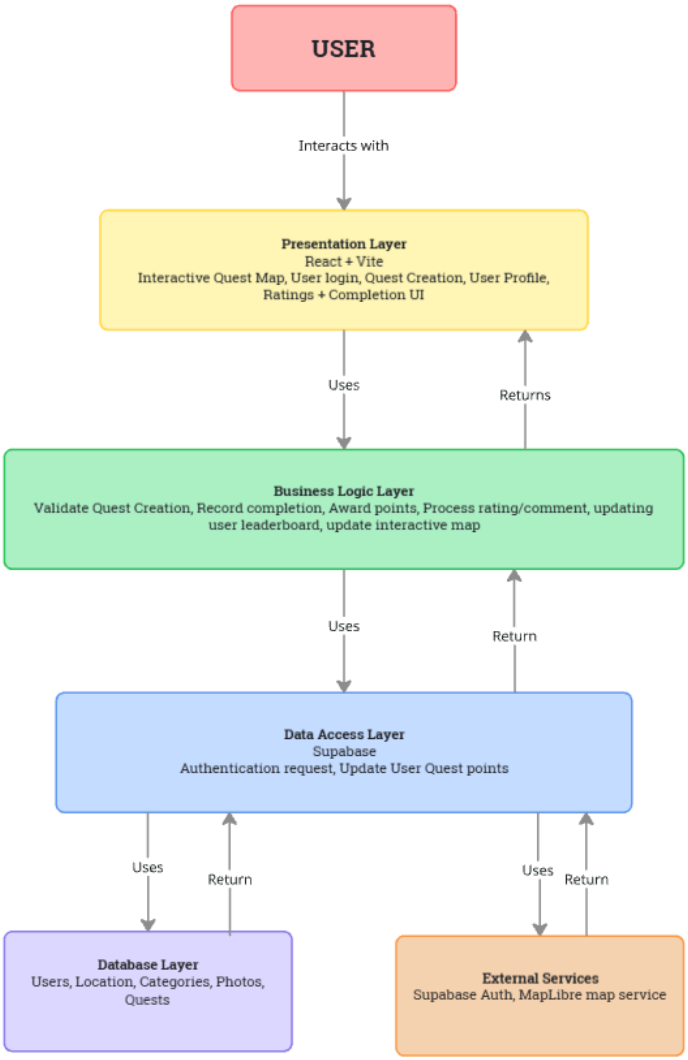
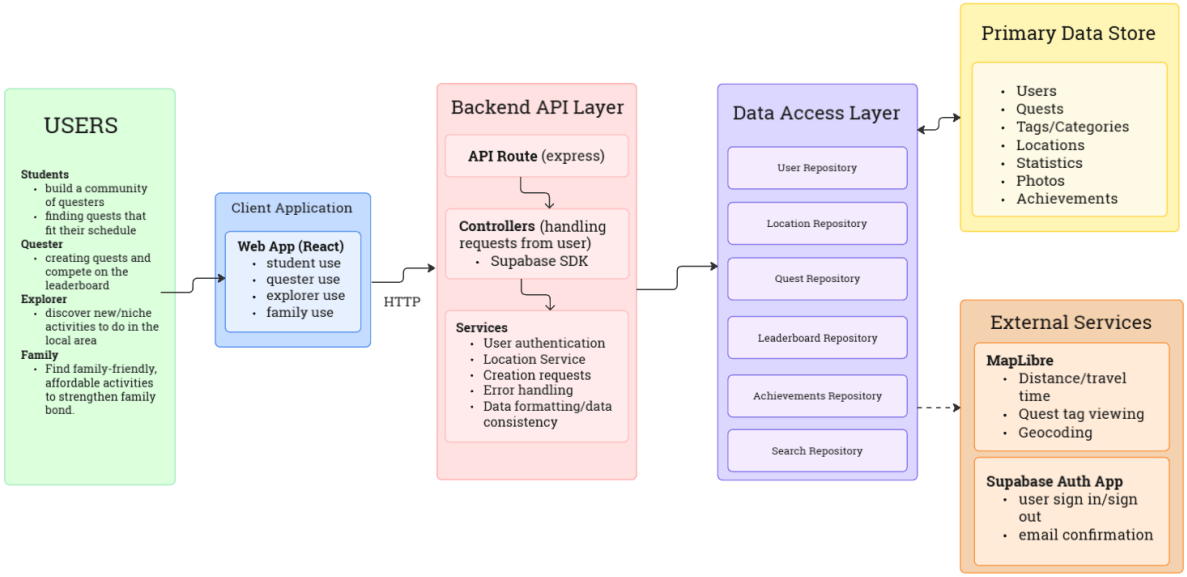
Part 1: Software architecture pattern

For our Software Architecture pattern, we decided to go with a Layered Architecture pattern. This fits our application because our web app will have different responsibilities that we do not want to overlap, and having them presented in layers will make it easy to maintain and test later on. Displaying user data, updating the leaderboard, and communicating with the Supabase should not happen simultaneously, so splitting the requirements between the layers creates a line of single responsibility that creates a seamless workflow.

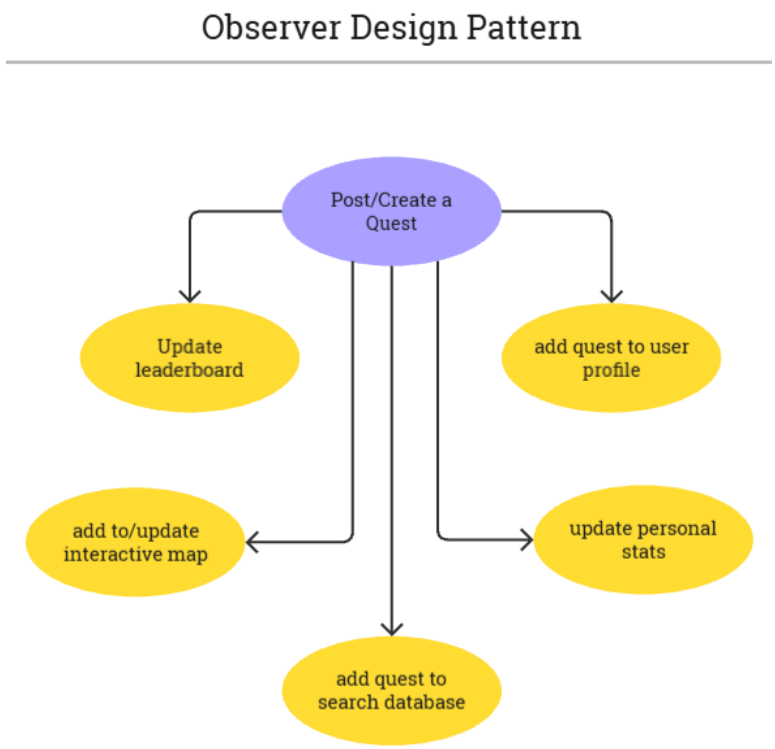


Part 2:



Part 3: Design pattern

We plan on using the observer pattern because we want to keep user information updated at all times. We don't want user data to stay static, so the observer pattern solves this issue best. When a user creates a quest, it should update the global leaderboard, user profile, user statistics, interactive map, and explore page. This pattern suits our project vision because it makes data easier to update and increases usability for users.



Part 4: Design Decisions and Tradeoffs

We decided to use Supabase Auth rather than building our own. This choice fits our project because it simplifies the process of creating authentication and allows us to seamlessly integrate the functionality of the app into our project.

We decided to use MabLibre instead of other map APIs like Google Maps, Mapbox. This choice best fits our project as MapLibre allows for further design and customization compared to other tools. We also chose this map API because it is completely open-source so all map rendering is completely free which is very useful when testing and modifying UI.